

Session 2: Values, names, and expressions

What values are, what their types allow, and why the computer's arithmetic sometimes surprises us.

Data Science and Advanced Programming · UNIL · 2026

ADMIN

We meet again tomorrow

The 21 September holiday is made up **tomorrow, Tuesday 29 September**, in **Internef 263**.

Lecture	10:15 – 12:00
----------------	---------------

Exercise session	12:30 – 14:00
-------------------------	---------------

Tomorrow is session 3: **lists, loops, and** `if` .

position.py

```
1 # position.py - see position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

4 shares, from ledger.csv

Double it

Last time, Python surprised us with `0.1 + 0.2`.

Today's first surprise is simpler.

The ledger says: **NESN, 4 shares**. Double it.

What does Python say?

```
$ python position.py
not run yet
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Double it, twice

Python gives two answers

`raw * 2` gives 44. `shares * 2` gives 8.

Both look like 4. No error either way.

The difference is not the number. It is the **type**. That is today: what kinds of values exist, and what each operator does to each kind.

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
```

Two different answers

```
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

```

position.py
1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4"           # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6   # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```

$ python position.py
not run yet

```

Today: one script, every line

The script on the left values **one position of the ledger**: NESN, 4 shares.

It stays on screen all lecture. Each part of today lights up the lines it explains.

1. Values and their types
2. Names: beyond `=`
3. Arithmetic
4. Floats in practice
5. The type decides what an operator does
6. When a line explodes
7. Comparisons and booleans
8. Printing values: f-strings

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```

$ python position.py
not run yet

```

Read it cold

Take two minutes. Read the script on your own.

- Which lines do you understand already?
- Which lines are a mystery?
- **Guess the output.** How many lines will it print?

Write your guess down. We check it in a moment.

```

position.py
1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4"
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Set up: what we own

Buy, then compute

Check the position

Report

The shape of the script

Four blocks, separated by empty lines:

Lines	Job
2–8	set up: what we own
10–16	buy 6 more shares, compute
18–19	check the position
21–25	report the results

Empty lines do nothing. They help the **reader**, not the machine.

```

$ python position.py
not run yet

```

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"           # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6   # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

What it prints

You do not need to run anything now. Bottom left is the output of one run of the script.

Six lines of output. The script has **six** `print` calls.

Did you guess right?

Some of this output looks strange: `44`, `<class 'str'>`, `386.20000000000005`. Each strange thing is a lesson for today.

```

$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None

```


position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # The result, not the recipe
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

REMINDER

How code runs (session 1)

We use all of this today without explaining it again.

- Python runs a file **top to bottom**, one line at a time.
- `=` means **gets**. `shares = shares + 6` is an instruction, not an equation.
- A variable holds the **result**, not the recipe. Excel updates cells; Python does not.
- **Trace by hand**: one row per line, one column per name.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Gives back a new value

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
```

```
386.20000000000005 False True
```

Close, not exact

```
NESN: 10 shares, 1,023.00 CHF
```

```
cash 386.20, weight 0.73
```

```
3 79.30000000000005 1127.86
```

```
True True None
```

REMINDER

Floats and failures (session 1)

- Floats are **approximations**. $0.1 + 0.2$ is 0.30000000000000004 . Never compare money with `==`.
- `round()` gives back a new value. Keep it with `=`, or it is lost.
- Code fails in three ways: it **won't start**, it **crashes**, or it **runs and lies**.
- Read an error from the **last line** first.

Today we go further on each of these.

```

position.py
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4"
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

A whole line

text, as read from a file

After code

Says why

```

$ python position.py
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Comments: notes for humans

starts a **comment**. Python ignores everything from # to the end of the line.

```

# a whole line of comment
print("NESN") # a comment after code
# print("switched off")

```

NESN

Code says **what** the machine does. A comment says **why**, for the next reader: a colleague, or you in a month.

Good: # buy 6 more . Bad: x = 5 # set x to 5 repeats the code.

In VS Code, **Ctrl+/ **(Mac: **Cmd+/**) switches a line into a comment and back.********

PART 1

Values and their types

Every value has a type. The type decides what you can do with it.

```

position.py
1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4" # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = rest > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Every value has a type

A **value** is a piece of data: "NESN", 4, 102.3.

Every value has a **type**. The type says what the value is, and what you can do with it.

Session 1: bits mean nothing until a program decides what they are. **A type is that decision.**

Type	What it holds	On the left
str	text	"NESN"
int	whole numbers	4
float	numbers with a decimal point	102.3
bool	True or False	line 18
NoneType	None : "no value"	line 7

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = 10
5 price = 1023.0
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Quotes make text

Text too, not a number

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

str: text

A **string** (`str`) is text between quotes: `"NESN"`.

Single or double quotes both work: `'NESN'` is the same string.

The quotes are not part of the text. They mark where the text starts and ends.

`"4"` **is text**. It holds the character 4, not the number 4. It looks like a number to you. To Python it is text.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw) # An int: 4, then 10
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

int: whole numbers

An **integer** (`int`) is a whole number: `4`, `10`, `-3`, `0`.

No decimal point. No quotes.

Integers are **exact**. `4 + 6` is exactly `10`, always.

Shares are counted in whole numbers, so `shares` is an `int`.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = 4
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Decimal point: float

Also a float: it has a point

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

float: numbers with a decimal point

A **floating-point number** (`float`) has a decimal point:

102.3, 1000.0.

1000 and 1000.0 are equal in value, but they have **different types**:

```
print(type(1000))
print(type(1000.0))
```

```
<class 'int'>
<class 'float'>
```

Floats are the numbers stored only approximately (session 1). Part 4 puts that to work.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3 # 1.023 × 102
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # 1023.0 = 1.023 × 103
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

REMINDER

Why "float"? The point floats

Seen in session 1. `float` is short for **floating point**:

value	1	0	2	3	.	= 1.023 × 10 ³
price	1	0	2	3		= 1.023 × 10 ²
	1	0	2	3		= 1.023 × 10 ¹
	1	0	2	3		= 1.023 × 10 ⁰

Same digits. Only the **point** moves, and the power of ten says where.

1	11 bits	52 bits
sign	exponent : where the point goes	fraction : the digits, in base 2 (about 15 decimal digits)

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

True or False

valid = len(ticker) <= 5 and shares > 0

alert = weight > 0.5 or not valid

True or False

```
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

\$ python position.py

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
```

True True None valid, alert

bool: True or False

A **boolean** (`bool`) has only two values: `True` and `False`.

Capital letter, no quotes. `true` and `"True"` are something else.

Booleans are **answers to yes/no questions: Is the data valid? Should we raise an alert?**

Lines 18 and 19 compute two booleans. Part 7 shows how.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None We have not sold yet
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

None : no value (yet)

None means "nothing here".

We have never sold NESN, so there is no last sale.

`last_sale = None` says exactly that.

None is not 0. A sale at price 0 would be a real (and strange) sale. None means no sale at all.

It has its own type, `NoneType`, with one value: `None`.

\$ python position.py

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None Printed as None
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Ask for the type

`type(raw)`, `type(shares)`

type() : ask Python

When you are not sure, **ask**. `type(x)` gives the type of `x`.

```
print(type("NESN"), type(102.3))
print(type(True), type(None))
```

```
<class 'str'> <class 'float'>
<class 'bool'> <class 'NoneType'>
```

`<class 'str'>` is Python's way to say "this is a `str`".

This is the most useful question when code behaves strangely.

\$ python position.py

```
44 8 <class 'str'> <class 'int'> str and int
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

PART 2

Names: beyond

You know what  does. Now: trace it yourself, write it shorter, and choose good names.

position.py

```
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Your turn: trace it

Session 1's tracing table, on today's script. Write the value after each line:

After line	shares	cash
4	?	?
6	?	?
10	?	?
11	?	?

Two minutes. Then compare with your neighbour.

```

position.py
1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4" # text, as read from a file
4  shares = int(raw) shares: 4
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9  shares: 10
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Check your trace

After line	shares	cash
4	4	—
6	4	1000.0
10	10	1000.0
11	10	386.20000000000005

Every later line sees the **latest** value. Line 22 prints 10 shares, not 4.

Tracing by hand is the main skill for the midterm. We practise it every week.

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None

```

Line 22 sees the new value

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price # cash = cash - 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Augmented assignment: `-=`, `+=`

`cash -= 6 * price` is short for `cash = cash - 6 * price`.

The same shortcut exists for every operator:

```
shares = 4
shares += 6
shares *= 2
print(shares)
```

```
20
```

`+=` is very common in real code. When you read it, say it long: **shares gets shares plus 6**.


```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4"           # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None    Words joined by _
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6  # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 2y = value * 1.05 ** 2 Starts with a digit
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```
File "position.py", line 16
  2y = value * 1.05 ** 2
  ^
```

```
SyntaxError: invalid decimal literal
```

Naming rules

A name can use letters, digits, and `_`. It **cannot start with a digit**: line 16 renamed to `2y`, and the program does not even start.

Case matters: `price` and `Price` are two different names.

By convention, Python names are lowercase with `_` between words: `last_sale`, `can_buy`. This is called **snake_case**.

A good name says what the value **is**. `x` says nothing. `cash` says everything.

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None # snake_case
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price) # snake_case
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

+ GOOD TO KNOW

Other naming conventions

Style	Example	Used for
snake_case	last_sale	Python names (this course)
UPPER_CASE	MAX_WEIGHT	Python constants
PascalCase	DataFrame	Python classes (pandas)
camelCase	lastSale	JavaScript, Java
kebab-case	last-sale	file names, web addresses

kebab-case never works in Python: – means minus.

```
last-sale = None
```

```
SyntaxError: cannot assign to expression here. Maybe
you meant '==' instead of '='?
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

+ GOOD TO KNOW

One project, one convention

Pick one convention per project, and never mix.

`lastSale`, `last_sale` and `LastSale` in one file make the reader stop and wonder: are these three different things?

In companies, this is not a choice. Programmers **must** follow a written **style guide**, so that anyone on the team can read and change anyone's code.

For Python, the standard guide is **PEP 8**. Tools check it automatically before new code is accepted.

In this course: `snake_case` for names. Every example follows PEP 8.

PART 3

Arithmetic

Seven operators, one order of operations,
and parentheses for when in doubt.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The arithmetic operators

Operator	Meaning	Example	Result
+ -	add, subtract	7 - 2	5
*	multiply	7 * 2	14
/	divide	7 / 2	3.5
//	floor division	7 // 2	3
%	remainder	7 % 2	1
**	power	7 ** 2	49

```
print(7 - 2, 7 * 2, 7 / 2, 7 // 2, 7 % 2, 7 ** 2)
```

```
5 14 3.5 3 1 49
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # int × float = float
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF # 10 × 102.3
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Mixing int and float

`shares` is an `int` (10). `price` is a `float` (102.3).

`shares * price` gives 1023.0, a **float**.

The rule: if one side is a `float`, the result is a `float`.

```
print(10 * 102.3)
print(10 * 2)
```

```
1023.0
20
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = cash // price
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Share of all my money in NESN

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73 73%
3 79.300000000000005 1127.86
True True None
```

/ always gives a float

Line 13 computes the **weight**: how much of my money is in NESN.

/ always returns a float, even when the division is exact:

```
print(10 / 5)
print(type(10 / 5))
```

```
2.0
<class 'float'>
```

2.0, not 2. Remember this when you expect a whole number.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / cash
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

How many whole shares fit?

can_buy

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86 can_buy
True True None
```

// : floor division

How many **more** shares can I buy with my cash? Only whole shares.

cash // price divides and **drops the decimals**: 386.2 / 102.3 is about 3.8, so the answer is 3.0.

// on floats gives a float. Line 14 wraps it in int() to get the int 3.

```
print(386.2 // 102.3)
print(int(386.2 // 102.3))
```

```
3.0
3
```


position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price # What is left over
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86 rest
True True None
```

%: the remainder

% gives what is **left over** after // .

If I buy 3 more shares, 79.30 CHF stays in cash.

```
print(7 // 2, 7 % 2)
print(10 // 3, 10 % 3)
```

```
3 1
3 1
```

// and % are a pair: **how many times does it fit, and what is left?**

(And yes: 79.300000000000005 again. Part 4.)

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

+5% per year, 2 years

1.05 ** 2

\$ python position.py

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86 in_2y, rounded
True True None
```

** : power

`1.05 ** 2` is 1.05×1.05 : two years of 5% growth.

```
print(1.05 ** 2)
print(1023.0 * 1.05 ** 2)
```

```
1.1025
1127.8575
```

Careful: `^` is **not** power in Python. It runs, with no error, and gives a different answer:

```
print(2 ** 3, 2 ^ 3)
```

```
8 1
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = (can_buy <= 5 and shares > 0)
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

1st: power

2nd: multiply

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Order of operations

Python uses the order from school:

1. `**` power
2. `*` `/` `//` `%`
3. `+` `-`

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

```
14
20
```

Line 16: first `1.05 ** 2`, then multiply by `value`. Line 11: first `6 * price`, then subtract.

```

1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / value + cash
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Parentheses removed

Parentheses change the meaning

On the left, line 13 has lost its parentheses.

Now `value / value` runs first (that is `1`), then `+ cash`: a "weight" of 387.20.

No error. A wrong number. **When in doubt, add parentheses.** They cost nothing.

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 387.20
3 79.300000000000005 1127.86
True True None

```

A weight of 387?

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Part 3 in one table

Line	Expression	Value	Type
11	6 * price	613.8	float
12	shares * price	1023.0	float
13	value / (value + cash)	0.7259...	float
14	int(cash // price)	3	int
15	cash % price	79.300000000000005	float
16	value * 1.05 ** 2	1127.8575	float

An **expression** is any piece of code that produces a value. Every right side here is one.

PART 4

Floats in practice

Session 1 showed that floats are approximate. Now: what to do about it.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3 # Stored as 102.2999... (session 1)
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

So this is False

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
```

```
386.20000000000005 False True # The error shows up
```

```
NESN: 10 shares, 1,023.00 CHF
```

```
cash 386.20, weight 0.73
```

```
3 79.30000000000005 1127.86
```

```
True True None
```

REMINDER

Session 1's floats, in our ledger

Last time: 102.30 is stored as 102.2999... . Now it shows up in our cash:

```
print(1000.0 - 6 * 102.3)
```

```
386.20000000000005
```

So `cash == 386.2` is `False`. Session 1 said: never compare money with `==`.

What do we do instead?

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Close enough?

\$ python position.py

```
44 8 <class 'str'> <class 'int'>
```

```
386.20000000000005 False True
```

Yes, within 1 cent

```
NESN: 10 shares, 1,023.00 CHF
```

```
cash 386.20, weight 0.73
```

```
3 79.30000000000005 1127.86
```

```
True True None
```

Compare with a tolerance

Ask a better question: **is the difference smaller than one cent?**

```
abs(cash - 386.2) < 0.01
```

`abs(x)` is what math writes as **|x|**: the size of a number without its sign.

```
cash = 1000.0 - 613.8
print(abs(-3.5), abs(3.5))
print(cash == 386.2, abs(cash - 386.2) < 0.01)
```

```
3.5 3.5
False True
```

The **tolerance** (here `0.01`) is your choice. For money, one cent is natural.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The rule for money

- Floats are fine for **computing**: growth, weights, averages.
- **Never** compare floats with `==`. Use a tolerance.
- **Round only for display**, at the very end.
- Serious accounting systems count **cents as int**:
`38620` cents is exact.

```
cash_cents = 100000 - 6 * 10230
print(cash_cents, cash_cents == 38620)
```

```
38620 True
```

PART 5

The type decides what an operator does

The same `*` multiplies numbers and repeats text. The same `+` adds and joins.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

str * int

int * int

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'> Two meanings of *
```

```
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

Back to the puzzle: 44 and 8

The operator `*` means different things for different types:

- **number** `*` **number**: multiply. `4 * 2` is `8`.
- **text** `*` **number**: repeat the text. `"4" * 2` is `"44"`.

```
print("4" * 2)
print("-" * 20)
```

```
44
-----
```

Python looks at the **types** of the values, then decides what to do.

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file Text!
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

+ on text joins

For strings, + **joins** them. This is called **concatenation**.

```
print("3" + "4")
print("NESN" + " " + "shares")
```

```
34
NESN shares
```

"3" + "4" gives 34, not 7. **No error.**

The machine did exactly what we asked. What we asked was wrong.

```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = raw + 6    # str + int
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6  # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Text + number: Python refuses

Line 4 now adds 6 to raw, which is the text "4".

Should "4" + 6 be 10 or "46"? Python **does not guess**. It stops with a `TypeError`.

This error is a friend. It is loud, and it tells you the two types that did not fit.

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
File "position.py", line 4, in <module>
```

```
    shares = raw + 6
```

```
~~~~^~~
```

```
TypeError: can only concatenate str (not "int") to str
```

position.py

```

1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw) Turn "4" into 4
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

\$ python position.py

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Converting between types

Line 4 converts text to a number: `int(raw)` turns "4" into 4.

Call	Gives
<code>int("4")</code>	4
<code>float("102.3")</code>	102.3
<code>str(10)</code>	"10"
<code>int(4.9)</code>	4 (cuts, does not round)

```

print(int("4") + 6, float("102.3") * 2)
print(str(10) + "0", int(4.9))

```

```

10 204.6
100 4

```

```

1  # position.py - one position of the ledger
2  ticker = "AAPL"
3  raw = "4.5" # text, as read from a file
4  shares = int(raw) # int() refuses
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Conversion can fail

`int()` only accepts text that looks like a **whole** number. On the left, line 3 now holds `"4.5"`, and line 4 stops the program.

A `ValueError`: the type was right (text), but the **value** made no sense for `int()`.

The same happens with `int("four")`. Part 6 comes back to this report and shows how to read it.

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
  File "position.py", line 4, in <module>
```

```
    shares = int(raw)
```

```
          ^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: '4.5'
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw) # You convert: explicit
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # Python coerces
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Conversion you ask for, and conversion you don't

- **Explicit conversion:** you call `int()`, `float()` or `str()`. Line 4.
- **Type coercion:** Python converts a value **by itself**, to make an operation work. Line 12.

Line 12 mixes an `int` and a `float`. Watch what Python does, step by step.

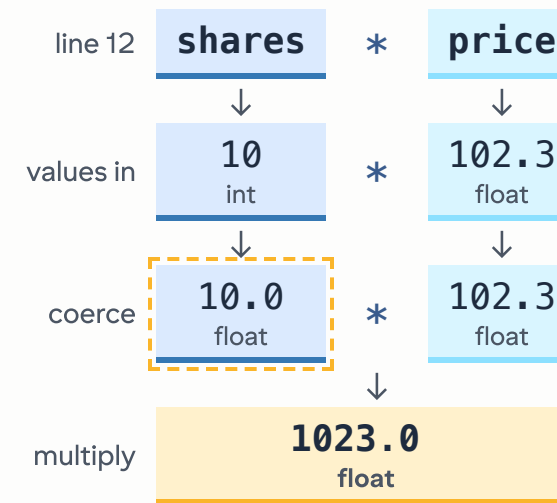
```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```


position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # int * float
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Coercion, step by step: line 12



Python **coerces** `10` into `10.0` for you (dashed box), then multiplies two floats. You never see the step. You only see its result: a `float`.

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF # A float, formatted
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # int meets float
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

What happens in a mix

Line 12 is one case. Here are the others. When two different types meet, Python coerces one of them first:

You write	Python coerces	Result
True + 10	True → 1	11 (int)
10 * 102.3	10 → 10.0	1023.0 (float)
True * 102.3	True → 1.0	102.3 (float)
"4" + 6	nothing	TypeError

Line 12 is the second row. Is there a pattern? Next slide.

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price # int → float, then multiply
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The rule behind the table

Every row of the table follows one path, **towards** float:



Each arrow is a coercion from the table: Python does it **by itself** when two types meet. It never goes back: 102.3 as an int would lose the .3. str has no arrow: Python never coerces text.

So to predict a result: find both types on the path. The answer has the type **further right**.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
```

```
True True None valid, alert
```

bool is a number too

At the bottom of the ladder: `True` counts as `1`, `False` as `0`.

```
valid = True
alert = True
print(True + True, True * 10)
print(valid + alert)
```

```
2 10
2
```

`valid + alert` is `2`: the number of flags that are `True`. Adding up booleans is a quick way to count.

position.py

```
1 # position.py - see position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw) # int(raw) == 4 is True
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Numbers coerce, text never does

The same rule holds for comparisons:

```
raw = "4"
print(1023 == 1023.0)
print(raw == 4)
print(int(raw) == 4)
```

```
True
False
True
```

`1023 == 1023.0` is `True`: the `int` is coerced to a `float`.

`raw == 4` is `False`, **with no error**. Text and numbers are simply never equal. A check like this runs and lies: convert first, then compare.

position.py

```
1 # position.py - see position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Text that looks like a number

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

+ GOOD TO KNOW

Other languages coerce more

JavaScript, the language of web pages, converts text **whenever it can**:

JavaScript	gives	Python
"4" + 6	"46" (text)	TypeError
"4" * 2	8	"44"
"4" == 4	true	False

Same symbols, three different surprises. Python's choice is fewer conversions: a **loud error** instead of a quiet guess. When you move between languages, check how each one coerces.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

+ GOOD TO KNOW

Some languages never coerce

Here is our line 12 in **Swift**, Apple's language:

```
let shares = 10
let price = 102.3
let value = shares * price
```

```
error: binary operator '*' cannot be applied to
operands of type 'Int' and 'Double'
```

Swift refuses to mix whole numbers and decimals. You convert **by hand**: `Double(shares) * price` gives `1023.0`. Rust and Go work the same way.

Python sits in the middle: numbers coerce automatically, text never does.

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw) # Why raw is text
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Data from files arrives as text

When Python reads a file, every value is **text**: "4", "102.3", "NESN".

That is why line 3 holds "4" and not 4. It imitates a value read from ledger.csv.

Convert first, then compute. Line 4 does exactly this.

In session 6 we read the real file. This rule will matter on every line.

PART 6

When a line explodes

Session 1 promised it: how to read a traceback, and what to do next.

```

1  # position.py - one position of the ledger
2  ticker = "AAPL"
3  raw = "4.5" # text, as read from a file
4  shares = int(raw) # Explodes here
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

When line 4 explodes

Session 1 promised this. Change one value: the file now says **4.5** shares.

Run the script. Bottom left: instead of our six lines, Python prints a **report** and stops.

Nothing after line 4 runs. Not even the `print` on line 8.

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
File "position.py", line 4, in <module>
```

```
    shares = int(raw)
```

```
            ^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: '4.5'
```

```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4.5"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```
Traceback (most recent call last):
```

← The path, oldest step first

```
File "position.py", line 4, in
```

```
    shares = int(raw)
    ^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: '4.5'
```

The word "traceback"

To **trace back** is to follow a path backwards, step by step.

When a line fails, Python traces back **the path the program took to reach it**, and prints that path. The report is a **traceback**. Other languages call it a **stack trace**.

(most recent call last): the failing step comes **last**, at the bottom. That is why we read the last line first.

Today the path has one step: `<module>`, which means the top level of your file. From session 4, functions call functions, and the path gets longer.

```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4.5"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```

Traceback (most recent call last):
  File "position.py", line 4, in <module>
    shares = int(raw)

```

The kind of error ^^^^^^^

```
ValueError: invalid literal for int() with base 10: '4.5'
```

Reading a traceback, 1: what went wrong

Start at the **last line**:

```
ValueError: invalid literal for int() with base
10: '4.5'
```

- `ValueError` is the **kind** of error: the type was right (text), the value was not.
- After the colon come the **details**: `int()` got `'4.5'`, which is not a whole number. ("Base 10" means ordinary decimal numbers.)

Kinds so far: `NameError` (session 1), `TypeError`, `ValueError`.

```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4.5"          # text, as read from a file
4  shares = int(raw)    ← Line 4
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6   # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Reading a traceback, 2: where

One line up from the start: File "position.py", line 4.

- The **file**: which program failed. In a project with many files, this matters.
- The **line number**: where to look. Your editor numbers its lines, like the panel on the left.

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
File "position.py", line 4, in <module> ← Go to this line
```

```
    shares = int(raw)
    ^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: '4.5'
```

```

1  # position.py - one position of the ledger
2  ticker = "NESN"
3  raw = "4.5"          # text, as read from a file
4  shares = int(raw)    ← The failing piece
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6   # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Reading a traceback, 3: which piece

Python copies the line, and marks the failing piece with

```
^^^^.
```

Here it points at `int(raw)`, not at `shares =`. Storing was fine. The **conversion** failed.

On long lines, this marker saves you a lot of guessing.

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
File "position.py", line 4, in <module>
```

```
    shares = int(raw)
```

```
^^^^^^^^
```

← Points at int(raw)

```
ValueError: invalid literal for int() with base 10: '4.5'
```

```

1  # position.py - one position of the ledger
2
3  raw = "4.5"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```
Traceback (most recent call last):
```

```
File "position.py", line 4, in <module>
```

```
    shares = int(raw)
            ^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: '4.5'
```

What to do when a line explodes

1. Read the **last line**: the error type and message.
2. Go to the **file and line** it names.
3. On that line, **check every name**: its value and its type.
4. Fix the **cause**, then run again.

```
raw = "4.5"
print(raw, type(raw))
```

```
4.5 <class 'str'>
```

`int()` cannot turn `4.5` into a whole number. But ask first: can you own 4.5 shares? The crash may be protecting you from **bad data**.

```

1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = raw # The real mistake: no int()
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more # Noticed here
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```
44 44 <class 'str'> <class 'str'> raw and shares: both str
```

```
Traceback (most recent call last):
```

```
File "position.py", line 10, in <module>
```

```
    shares = shares + 6 # buy 6 more
```

```
~~~~~^~
```

```
TypeError: can only concatenate str (not "int") to str
```

The crash line is not always the wrong line

Line 4 forgot `int()`. Python did not complain: storing text is fine.

The crash comes on **line 10**, where text meets `+ 6`.

The traceback shows where the program **noticed** the problem. Trace back to where the wrong value **came from**: line 4.

The clue was already on screen: line 8 printed `<class 'str'>` for `shares` too.

PART 7

Comparisons and booleans

Questions with a yes-or-no answer, and how to combine them.

position.py

```

1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

10 > 0 → True

\$ python position.py

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

A comparison produces a value

`shares > 0` is a question. Its answer is a **value**: `True` or `False`.

Operator	Question
<code>==</code>	equal?
<code>!=</code>	not equal?
<code><</code> <code><=</code>	less (or equal)?
<code>></code> <code>>=</code>	greater (or equal)?

```

shares = 10
print(shares > 0, shares == 4, shares != 4)

```

True False True

```

1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw + " shares * 2 = " + str(2 * int(raw)) + " shares")
9
10 shares == shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Asks, and throws the answer away

= stores, == asks

- `shares = 10` **stores** 10 under `shares`.
- `shares == 10` **asks** whether `shares` is 10.

On the left, line 10 now **asks** instead of storing. The answer (`False`) is thrown away, `shares` stays 4, and the report says 4 shares, although the cash was spent on 6 more.

No error. One character, a silent lie.

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 4 shares, 409.20 CHF
cash 386.20, weight 0.51
3 79.300000000000005 451.14
True True None

```

Still 4

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

len(): the length of a string

`len(ticker)` counts the characters in `ticker`.

```
print(len("NESN"), len("A"), len(""))
```

```
4 1 0
```

Line 18 checks that the ticker has at most 5 characters:

`4 <= 5` is `True`.

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Both must be True

```

$ python position.py
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None valid

```

and: both must be true

A and B is True only when both are True.

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Line 18: the ticker is short enough **and** we own some shares. Both hold, so `valid` is True.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Flips it

At least one

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

or and not

A or B is True when **at least one** is True.

not A flips: not True is False, not False is True.

```
print(True or False, False or False)
print(not True, not False)
```

```
True False
False True
```

position.py

```

1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = (weight > 0.5) and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

1st: compare

2nd: not

\$ python position.py

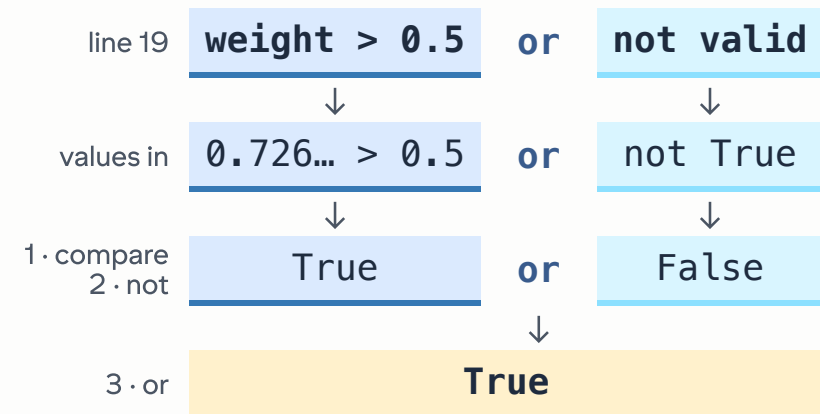
```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Reading line 19

Order: comparisons first, then **not**, then **and**, then **or**.



In plain English: **alert if more than half my money is in one stock, or the data is not valid.** Parentheses say the same thing more clearly: `(weight > 0.5) or (not valid)`.

PART 8

Printing values: f-strings

Put values inside text, and decide exactly how numbers look.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

f-strings: values inside text

Session 1 used commas in `print`. An **f-string** builds the text exactly as you want it. Line 22:

```
f"{ticker}: {shares} shares, {value:,.2f} CHF"
```

↓
NESN

↓
10

↓
1,023.00

prints

```
NESN: 10 shares, 1,023.00 CHF
```

- `f` before the quote switches the braces on.
- Each `{...}` is **replaced by a value**. Everything else is copied as it is.
- Without the `f`, the braces are printed as they are:
`{ticker}: {shares} shares`.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2))
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, in_2y, 2)
25 print(valid, alert, last_sale)
```

The format

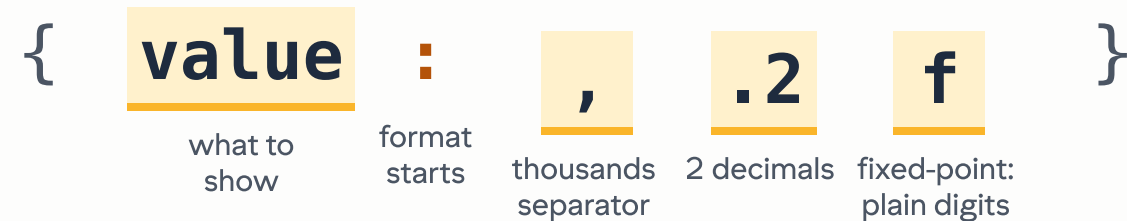
The format

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The format spec, piece by piece

After the name, `:` starts a **format spec**: how the value should look.



prints `1,023.00`

The last letter says **how to write the number**:

Spec	Gives	Meaning
<code>{value:.2f}</code>	<code>1023.00</code>	fixed-point: plain digits
<code>{value:.2e}</code>	<code>1.02e+03</code>	exponent: 1.02×10^3
<code>{value:.2}</code>	<code>1e+03</code>	no letter: 2 significant digits

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

What f-strings can do

You write	You get	What for
<code>f"{value}"</code>	1023.0	the plain value
<code>f"{value:.2f}"</code>	1023.00	money: 2 decimals
<code>f"{value:,.2f}"</code>	1,023.00	big numbers
<code>f"{value:.0f}"</code>	1023	no decimals
<code>f"{weight:.1%}"</code>	72.6%	× 100, then add %
<code>f" {ticker:>6} "</code>	NESN	align right, width 6
<code>f" {ticker:<6} "</code>	NESN	align left, width 6
<code>f"{shares * price:,.2f}"</code>	1,023.00	any expression
<code>f"{value=}"</code>	value=1023.0	checking a value

`{value=}` prints the name **and** the value: the quickest way to check a variable while you debug.

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.000000000000000005)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

Read it: value, commas, 2 decimals, plain digits

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.20000000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000000005 1127.86
True True None
```

+ GOOD TO KNOW

You don't need to memorize format specs

Nobody remembers them all. They are **easy to look up**:

- search for "**Python format specification**" (the official page is called "Format Specification Mini-Language"),
- or ask a chatbot: "f-string: show 1023.0 as 1,023.00".

What you **do** need: to **read** one when you meet it.

`{value:,.2f}` → the value of `value`, with thousands separators, 2 decimals, plain digits.

`{weight:.1%}` → the value of `weight`, as a percentage with 1 decimal.

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

What you see is not what is stored

The same variable `cash`, printed twice:

- line 21: `print(cash)` shows the stored value.
- line 23: `{cash:.2f}` shows a **rounded picture** of it.

Formatting changes the display. It does **not** change the value.

Money can look right on screen and still be a float underneath.

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
```

```
386.20000000000005 False True
```

Stored

```
NESN: 10 shares, 1,023.00 CHF
```

```
cash 386.20, weight 0.73
```

Displayed

```
3 79.30000000000005 1127.86
```

```
True True None
```

position.py

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The built-in functions of today

Python comes with functions you can use without doing anything:

Function	Does
<code>print(...)</code>	shows values
<code>type(x)</code>	the type of <code>x</code>
<code>int(x)</code> <code>float(x)</code> <code>str(x)</code>	convert
<code>len(s)</code>	length of a string
<code>abs(x)</code>	<code> x </code> : size without sign
<code>round(x, n)</code>	round to <code>n</code> decimals

You **call** a function with parentheses. Session 4 is all about functions.

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Three ways code fails: sort today's examples

Won't start, crashes, or runs and lies? Decide for each:

1. `"3" + "4"` gives `34`
2. `int("4.5")`
3. `2y = 5`
4. `value / value + cash`
5. `"4" + 6`
6. `shares == 10` where we meant `=`
7. `2 ^ 3` for "2 to the power 3"

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"           # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6   # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

```
$ python position.py
```

```

44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

Sorted

Example	Kind
2y = 5	won't start (<code>SyntaxError</code>)
int("4.5")	crashes (<code>ValueError</code>)
"4" + 6	crashes (<code>TypeError</code>)
"3" + "4" gives 34	runs and lies
value / value + cash	runs and lies
shares == 10 for =	runs and lies
2 ^ 3 for power	runs and lies

Four of seven run and lie. **Python checks the grammar. It never checks the meaning.**

position.py **CHANGED ON THIS SLIDE**

```
1 # position.py – one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash - can_buy * price
16 in_2y = value * 1 + 0.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

A classmate's version

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1023.0
True True None
```

Two years of growth?

Spot the lie

A classmate writes the growth line (16) from scratch.

It runs. Line 24 prints a value in two years that looks like money.

Is it right? Which kind of failure is this, and how would you catch it?

position.py

```
1 # position.py - one position of the ledger
2 ticker = "NESN"
3 raw = "4" # text, as read from a file
4 shares = int(raw)
5 price = 102.3
6 cash = 1000.0
7 last_sale = None
8 print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6 # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:,.2f}, weight {weight:,.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

The fix: grow by 1.05, twice

1.05 ** 2

The lie, explained

`**` runs first: `0.05 ** 2` is `0.0025`. The classmate's line adds a quarter of a cent, not two years of growth. Then `round(in_2y, 2)` on line 24 hides the quarter cent: it prints `1023.0`.

Catch it with a rough check: 5% of 1,000 is 50 a year, so expect about 1,100.

On the left, the original line 16: `value * 1.05 ** 2` gives `1127.86`. Two of today's lessons, precedence and rounding, hid one mistake together.

\$ python position.py

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

The right answer

```

position.py
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)

```

Read it again

Same script as the start of the lecture. Read it once more.

Every line is now yours:

- the **type** of every value,
- what every **assignment** stores,
- what every **operator** does,
- why each output line looks the way it does.

```

$ python position.py
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None

```

position.py

```
1  # position.py – one position of the ledger
2  ticker = "NESN"
3  raw = "4"          # text, as read from a file
4  shares = int(raw)
5  price = 102.3
6  cash = 1000.0
7  last_sale = None
8  print(raw * 2, shares * 2, type(raw), type(shares))
9
10 shares = shares + 6    # buy 6 more
11 cash -= 6 * price
12 value = shares * price
13 weight = value / (value + cash)
14 can_buy = int(cash // price)
15 rest = cash % price
16 in_2y = value * 1.05 ** 2
17
18 valid = len(ticker) <= 5 and shares > 0
19 alert = weight > 0.5 or not valid
20
21 print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
22 print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
23 print(f"cash {cash:.2f}, weight {weight:.2f}")
24 print(can_buy, rest, round(in_2y, 2))
25 print(valid, alert, last_sale)
```

```
$ python position.py
```

```
44 8 <class 'str'> <class 'int'>
386.200000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.300000000000005 1127.86
True True None
```

Check yourself

Answer on paper, then check with the person next to you:

1. What is the type of `can_buy` ? Of `rest` ?
2. What would `print(raw + raw)` show?
3. Line 3 becomes `raw = "5"`. What is `shares` after line 10?
4. What does `7 // 2 + 7 % 2` give?
5. What does `print(f"{rest:.2f}")` show?

What you can now do

- Name the type of any value, and check it with `type()`.
- Predict what `+` and `*` do to text and to numbers.
- Convert between text and numbers, and say when conversion fails.
- Use `//` `%` `**` and predict the order of operations.
- Compare floats with a tolerance; keep money exact in cents.
- Build conditions with `and`, `or`, `not`.
- Format numbers with f-strings.
- Read a traceback: find the line that exploded, and the line that caused it.

Next session: **tomorrow, Tuesday 29 September**: lecture 10:15–12:00, exercise session 12:30–14:00. **Lists, loops, and** `if`: one position becomes the whole ledger.